Nexus Report

Nexus: Enterprise-Grade Academic RAG Architecture

Technical Design & Post-Mortem Report

1. Executive Summary

Nexus is a fully deployed, full-stack, multi-agent Retrieval-Augmented Generation (RAG) system designed to synthesize high-difficulty, graduate-level academic assessments from raw course materials.

Standard LLM wrappers fail in academic contexts due to context-window bloat, mathematical truncation, and hallucinations. Nexus solves this by moving away from single-pass generation, implementing a dual-concurrency ingestion engine, a two-stage retrieval pipeline, and a fault-tolerant, 3-stage agentic loop orchestrated via Server-Sent Events (SSE).

2. Ingestion Engine: Resilient Data Processing

The ingestion pipeline is designed to handle massive, unstructured academic PDFs (e.g., 200MB+ of NLP lecture materials) without bottlenecking the server or breaking token limits.

- **Dual-Concurrency Architecture & Performance:** * **CPU-Bound:** A `ProcessPoolExecutor` parallelizes document parsing across multiple CPU cores using PyMuPDF4LLM. During load testing, this architecture successfully parsed, chunked, and embedded **22 complex NLP lecture PDFs (over 210MB of data) in under 6 minutes**, proving the parallelization effectively bypasses Python's Global Interpreter Lock (GIL).
    - **I/O-Bound:** A `ThreadPoolExecutor` (with a custom `botocore` configuration allowing 25 max pool connections) handles concurrent network requests to AWS Bedrock for embeddings without blocking the CPU parsers.
- **Semantic Markdown Chunking & Safety Nets:** Instead of arbitrarily slicing text by character count, the parser groups text by Markdown headers. To mathematically guarantee that chunks never exceed AWS Titan's 8,192 token limit, a hard 8,000-character truncation safety net is enforced before embedding.
- **Asynchronous Vision Fallback & Garbage Collection:** When the parser encounters diagrams, it routes them to a Vision LLM to generate a dense semantic description. A

custom regex cleaner actively strips out corrupted Base64 image leaks that frequently crash downstream embedding tokenizers.

3. Two-Stage Retrieval Pipeline

To maximize Context Precision while managing memory constraints, Nexus utilizes a two-stage retrieval mechanism rather than relying purely on Cosine Similarity.

- **Stage 1: Dense Retrieval (INT8 Quantized):** The system uses Qdrant with AWS Titan vectors. To optimize for a single-node deployment, vectors are compressed using INT8 Scalar Quantization (`always_ram=True`), heavily reducing the memory footprint while maintaining rapid search times.
- **Stage 2: Cross-Encoder Reranking:** The top broad chunks are passed to a local Cross-Encoder (`ms-marco-MiniLM-L-12-v2`). The reranker physically evaluates the user's query against the chunks, mathematically rescoring them to guarantee only the highest-fidelity context reaches the Generator.
- **Simulated Hybrid Search:** Users can pass explicit keywords into the Planner Agent, mathematically anchoring the initial Dense search and achieving near-BM25 accuracy without the overhead of maintaining a secondary sparse index.

4. Multi-Agent Orchestration & Multi-Chunk Synthesis

Nexus abandons the standard "chat" paradigm for a strict Planner-Generator-Critic agent loop powered by `zai.glm-5`, communicating via constrained JSON schemas validated by Pydantic.

- **Agent 1 (The Planner):** Interprets the user's intent, extracts search keywords, and writes strict boundaries for the Generator.
- **Agent 2 (The Generator):** Drafts the question using a forced Chain-of-Thought (Reasoning Scratchpad) to map distractors and explicitly cite source materials.
- **Agent 3 (The Critic & The 3-Strike Loop):** Acts as an independent quality gate. If a question is trivially easy or asks about administrative details, the Critic rejects it. The system automatically loops up to 3 times to replace the failure.

Overcoming Open-Weight Model Quirks (War Stories):
Initial prototypes were designed around Anthropic's Claude 3. However, due to AWS quota constraints and access limitations, the architecture was aggressively pivoted to utilize highly capable open-weight models available on Bedrock, specifically `moonshotai.kimi-k2.5` and `zai.glm-5`. Forcing these models to execute complex, multi-agent reasoning required significant custom resilience engineering:

- **The Degenerate Token Loop (Kimi's `!!!!!!!` Collapse):** When forcing the smaller Kimi model into strict AWS Tool Calling APIs for JSON output, its attention mechanism would occasionally collapse under the rigid schema constraints, resulting in infinite fallback token generation (e.g., looping `!!!!!!!` until hitting the max token limit). **Solution:** Completely ripped out the high-level orchestration libraries, reverted to raw text prompting, and built a custom Regex JSON parser to safely extract payloads from raw markdown responses.
- **The Multi-Chunk Synthesis Bottleneck:** Initially, the system constrained the Generator to citing a single chunk. When the LLM attempted highly complex, multi-document synthesis (e.g., comparing RNNs to Transformers across different lectures), the Critic falsely flagged it as a hallucination. **Solution:** Upgraded the Pydantic schema to accept an array of `source_chunk_ids`, concatenated the text in the orchestrator, and forced the Critic to evaluate against the combined context.
- **The Trailing Comma Crash:** The open-weight models frequently generated invalid JSON (e.g., trailing commas in arrays), crashing standard `json.loads()`. **Solution:** Implemented a forgiving Regex shield (`re.sub(r',\s*([\]}])', r'\1', json_str)`) to sanitize output on the fly.
- **Critic-Induced Mode Collapse & Context Stagnation:** Because the Critic agent's rejection rubric was so ruthlessly strict, the Generator learned to "reward hack" by defensively defaulting to a single, easily verifiable format (Roman Numeral True/False) and relying on the exact same safe chunks for every question. **Solution:** Injected dual-entropy into the orchestration loop. "Prompt Entropy" was introduced to randomly assign specific question styles (e.g., diagnostic, scenario-based) while explicitly banning Roman numerals. Simultaneously, "Context Shuffling" randomly subset the retrieved vector context on each iteration, physically forcing the Generator to synthesize diverse lecture topics.

5. Observability: Real-Time SSE & The Quarantine Zone

A core tenet of Nexus is observability.

- **Server-Sent Events (SSE):** The FastAPI backend streams the orchestration progress (Planner strategies, vector query counts, and Agent approvals/rejections) directly to the React UI in real-time, eliminating perceived loading latency during the computationally heavy generation loops.
- **The Quarantine Zone:** Instead of silently dropping failed generations, questions that fail the Critic's rubric 3 times are pushed to the frontend's Quarantine Zone. This provides users with absolute transparency into the AI's internal guardrails, displaying exactly what was blocked and why the Critic rejected it.

6. Deployment Infrastructure

The system is deployed as a robust, scalable architecture using Docker Compose.

- **Compute:** Hosted on an AWS EC2 `m6a.xlarge` instance. The 4 vCPUs and 16GB of RAM provide the sustained, unthrottled memory buffer required to run the local Qdrant vector database, the Cross-Encoder, and parallel CPU-bound PyMuPDF workers simultaneously without Out-Of-Memory (OOM) crashes.
- **Production State Management:** Background ingestion jobs survive process restarts via a localized SQLite database (`jobs.db`) mounted securely as a Docker volume to the host machine.
- **Networking & Security:** Secured via Caddy acting as a reverse proxy on ports 80/443, handling automated Let's Encrypt SSL certificates and routing subdomain traffic directly to the internal Docker network. AWS Bedrock credentials are not hardcoded; the system securely inherits `boto3` permissions via an attached IAM EC2 Role.

7. Future Optimizations (Roadmap)

With SSE and robust state management currently in production, the architecture has clear paths for further enterprise scaling:

- **UI Decoupling & Role-Based Access Control (RBAC):** Transitioning the React frontend from conditional tab rendering to strict `react-router-dom` URL paths (e.g., `/ingest` and `/generate`). This isolates heavy browser state and allows administrative authentication guards to be placed over the CPU-intensive ingestion routes.
- **True Hybrid Search:** Upgrading the simulated keyword anchoring to a true Dual-Index Qdrant implementation (Dense + BM25 Sparse with Reciprocal Rank Fusion) for ultimate namespace precision.